

Politecnico di Bari

Corso di Laurea in Ingegneria Informatica

PlantChain: Certificazione su Blockchain delle Varietà Vegetali Protette

Ingegneria del Software

Docente: Marina Mongiello

Autori:

Oronzo Franchini
Pasquale Lorusso
Giovanni Pastore

Anno Accademico

2025/2026

Indice

1	Introduzione	3
1.1	Contesto e ambito applicativo	3
1.2	Problema affrontato	4
1.3	Obiettivo del progetto	4
1.4	Approccio e struttura del lavoro	5
1.5	Motivazioni e valore aggiunto della soluzione proposta	5
2	Stato dell'arte	7
2.1	Blockchain e proprietà intellettuale	7
2.2	Applicazioni della blockchain in agricoltura	8
2.3	Il nostro contributo nel contesto dello stato dell'arte	8
3	Metodologia di sviluppo e pianificazione	10
3.1	Metodologia adottata: Agile–Scrum semplificato	10
3.2	Pianificazione del progetto e diagramma di Gantt	12
3.3	Analisi dei rischi	12
3.4	Stima dei costi	12
4	Requisiti e casi d'uso	15
4.1	Requisiti funzionali (FURPS – F)	15
4.2	Requisiti non funzionali (URPS)	17
4.3	Casi d'uso dettagliati	19
5	Architettura e tech stack	22
5.1	Architecture overview (3 layer)	22
5.2	Tech stack dettagliato	23
5.3	Smart contract architecture	25
5.4	Pattern architetturali e sicurezza	25
6	Prototipo	27
6.1	Implementazione smart contract	27

6.2	Frontend – DApp	29
6.3	Sequence diagrams & eventi	30
7	Validazione e verifica	33
7.1	Test unitari smart contract	33
7.2	Test end-to-end e UI/UX	33
7.3	Analisi coverage e strumenti di test	34
8	Discussione	35
8.1	Scelte progettuali principali	35
8.2	Sfide incontrate e soluzioni adottate	36
8.3	Limitazioni e sviluppi futuri	36
9	Conclusioni e sviluppi futuri	38
9.1	Obiettivi raggiunti	38
9.2	Sviluppi futuri	39

Capitolo 1

Introduzione

1.1 Contesto e ambito applicativo

La protezione delle varietà vegetali rappresenta un tema di grande rilevanza sia dal punto di vista economico sia dal punto di vista giuridico. Lo sviluppo di nuove varietà richiede infatti investimenti significativi in termini di ricerca, sperimentazione e tempo, soprattutto nel settore agricolo, dove i risultati dell'innovazione si manifestano spesso solo dopo diversi cicli produttivi. Per questo motivo, i sistemi normativi nazionali e internazionali riconoscono ai costitutori (breeder) specifici diritti di esclusiva, finalizzati a tutelare l'innovazione e a garantire un adeguato ritorno economico degli investimenti sostenuti.

A livello internazionale, la tutela delle varietà vegetali è disciplinata da accordi come la Convenzione UPOV del 1991 e dall'Accordo TRIPS, che stabiliscono i principi fondamentali per il riconoscimento e la protezione dei diritti del costitutore. Tali strumenti normativi definiscono la varietà vegetale come un bene immateriale, distinto dal seme in quanto oggetto fisico, e riconoscono al breeder il diritto di autorizzarne la produzione, la commercializzazione e l'utilizzo tramite sistemi di licenza.

Parallelamente, negli ultimi anni la tecnologia blockchain si è affermata come infrastruttura abilitante per applicazioni che richiedono tracciabilità, integrità dei dati e verificabilità pubblica. In particolare, la blockchain è stata ampiamente studiata per la gestione di asset digitali, diritti di proprietà intellettuale e licenze, grazie alle sue caratteristiche di decentralizzazione, immutabilità e trasparenza. Queste proprietà rendono la blockchain un candidato naturale per affrontare alcune delle criticità presenti nei sistemi tradizionali di certificazione e gestione delle varietà vegetali protette.

1.2 Problema affrontato

I sistemi attualmente utilizzati per la gestione delle varietà vegetali protette presentano diverse criticità, soprattutto quando si considerano la complessità delle filiere agricole e la necessità di controlli distribuiti nel tempo e nello spazio. In particolare, emergono le seguenti problematiche:

- difficoltà nella verifica dell'autenticità dei lotti di sementi lungo la filiera produttiva e distributiva;
- mancanza di trasparenza nei processi di emissione, revoca e consultazione delle licenze di coltivazione;
- forte dipendenza da registri centralizzati, che possono essere soggetti a errori, ritardi o manipolazioni;
- costi elevati e inefficienze nei controlli ispettivi tradizionali;
- complessità nel collegare in modo affidabile il mondo fisico (produzione dei semi) con il mondo digitale (registri e certificazioni).

Queste criticità incidono negativamente sia sui breeder, che faticano a tutelare i propri diritti, sia sugli utilizzatori finali, che non dispongono sempre di strumenti semplici e affidabili per verificare la legittimità dei lotti di sementi acquistati. Inoltre, la mancanza di un sistema di tracciabilità completo favorisce fenomeni di contraffazione e utilizzo non autorizzato delle varietà protette.

1.3 Obiettivo del progetto

L'obiettivo di questo progetto è la progettazione e la realizzazione di un sistema software basato su tecnologia blockchain per la certificazione e la gestione delle varietà vegetali protette. In particolare, il sistema proposto mira a:

- registrare in modo immutabile le varietà vegetali protette e il relativo titolare del diritto;
- consentire ai breeder di emettere e revocare licenze di coltivazione attraverso smart contract;
- tracciare la produzione di lotti di sementi associati a licenze valide;

- permettere la verifica pubblica e trasparente dei lotti, anche da parte di utilizzatori finali, tramite identificativi univoci o QR code;
- introdurre meccanismi di controllo e validazione che colleghino i dati on-chain con verifiche effettuate nel mondo reale.

Il sistema si ispira a modelli accademici esistenti, come il concetto di Non-Fungible Plant Variety (NFPV), che propone la tokenizzazione delle varietà vegetali come asset non fungibili, e integra tali idee in una soluzione orientata alla gestione operativa delle licenze e dei lotti di produzione.

1.4 Approccio e struttura del lavoro

Dal punto di vista metodologico, il progetto adotta un approccio ingegneristico che combina analisi teorica, modellazione del sistema e progettazione di un prototipo software. In una prima fase viene analizzato lo stato dell'arte, facendo riferimento ai principali contributi accademici e normativi relativi alla blockchain e alla protezione delle varietà vegetali.

Successivamente, viene definito il modello di processo e l'insieme dei requisiti funzionali e non funzionali del sistema. La progettazione dell'architettura si basa su una separazione chiara tra componenti on-chain, responsabili della gestione delle informazioni critiche e immutabili, e componenti off-chain, dedicate all'interazione con l'utente e allo storage documentale.

Il funzionamento del sistema viene descritto attraverso diagrammi UML e flussi operativi, che illustrano i ruoli degli attori coinvolti e le principali dinamiche applicative. Infine, il lavoro si conclude con una discussione dei risultati ottenuti, dei limiti della soluzione proposta e dei possibili sviluppi futuri.

1.5 Motivazioni e valore aggiunto della soluzione proposta

La soluzione proposta in questo progetto si distingue dai sistemi tradizionali e da altre applicazioni blockchain esistenti per una serie di scelte progettuali mirate, orientate non solo alla tracciabilità tecnica, ma anche alla concreta applicabilità nel contesto agricolo reale.

Nei sistemi tradizionali di certificazione delle varietà vegetali, la gestione delle informazioni è generalmente affidata a registri centralizzati, spesso frammentati tra diversi enti, con limitate possibilità di verifica pubblica. In

tali contesti, l'utilizzatore finale non dispone di strumenti semplici per accertare l'autenticità di un lotto di sementi o la validità della licenza associata, mentre i controlli risultano costosi e difficilmente scalabili. La nostra soluzione supera questi limiti introducendo un registro distribuito e immutabile, in cui lo stato delle varietà, delle licenze e dei lotti rappresenta una fonte di verità condivisa e verificabile.

Rispetto ad altre proposte basate su blockchain per la protezione della proprietà intellettuale, il sistema progettato non si limita alla tokenizzazione dell'asset o alla sola registrazione del diritto, ma modella esplicitamente l'intero ciclo di vita della varietà vegetale: dalla registrazione iniziale, all'emissione delle licenze, fino alla produzione e verifica dei singoli lotti. In questo modo, la blockchain non viene utilizzata come semplice archivio, ma come infrastruttura attiva di coordinamento tra i diversi attori della filiera.

Un ulteriore elemento di miglioramento è rappresentato dall'introduzione di meccanismi di validazione graduata dei lotti, che tengono conto non solo dello stato formale delle licenze, ma anche di eventi successivi come revoche o scadenze. Questo approccio consente di gestire in modo più realistico le situazioni che si verificano nel mondo reale, evitando una visione binaria "valido/non valido" e fornendo all'utilizzatore finale informazioni più complete e contestualizzate.

Infine, la soluzione proposta integra in modo esplicito il ruolo degli ispettori come ponte tra mondo fisico e mondo digitale, affrontando uno dei principali limiti delle applicazioni blockchain, noto come *oracle problem*. L'intervento umano qualificato viene registrato e reso tracciabile on-chain, aumentando il livello di fiducia complessivo del sistema senza rinunciare ai benefici della decentralizzazione.

Capitolo 2

Stato dell'arte

2.1 Blockchain e proprietà intellettuale

Negli ultimi anni la tecnologia blockchain è stata oggetto di crescente interesse come strumento per la gestione della proprietà intellettuale. La blockchain può essere descritta come un registro distribuito e immutabile, in cui le informazioni vengono validate da una rete di nodi e rese pubblicamente verificabili. Queste caratteristiche la rendono particolarmente adatta alla registrazione di diritti e transazioni che richiedono integrità dei dati, trasparenza e tracciabilità nel tempo.

In ambito di proprietà intellettuale, diversi studi analizzano l'utilizzo della blockchain per la registrazione di copyright, brevetti, marchi e licenze. L'obiettivo principale è fornire una prova temporale certa dell'esistenza di un diritto e delle operazioni a esso associate, riducendo la dipendenza da intermediari centralizzati. In questo contesto, gli smart contract vengono spesso citati come strumenti in grado di automatizzare l'esecuzione di accordi di licenza, rendendo verificabili le condizioni di utilizzo di un bene immateriale.

Tuttavia, la letteratura evidenzia come l'uso della blockchain nella proprietà intellettuale presenti anche limiti significativi. In particolare, la registrazione on-chain di un diritto non garantisce automaticamente che tale diritto venga rispettato nel mondo reale. Questo aspetto risulta critico quando l'asset tutelato è collegato a processi produttivi fisici, per i quali è necessario un collegamento affidabile tra eventi digitali e fatti reali.

2.2 Applicazioni della blockchain in agricoltura

Nel settore agricolo, la blockchain è stata proposta principalmente come strumento per migliorare la tracciabilità delle filiere e la trasparenza delle informazioni lungo il ciclo di vita dei prodotti. Numerose applicazioni si concentrano sul tracciamento di prodotti agroalimentari, dalla produzione alla distribuzione, con l'obiettivo di garantire l'origine, la qualità e la sicurezza degli alimenti.

In questo ambito, la blockchain viene utilizzata per registrare eventi rilevanti come la produzione, il trasporto e la trasformazione dei prodotti, consentendo agli attori della filiera e agli utilizzatori finali di verificare le informazioni in modo indipendente. Queste soluzioni mostrano come la tecnologia possa aumentare la fiducia e ridurre le asimmetrie informative tra produttori, distributori e consumatori.

Tuttavia, quando l'attenzione si sposta dalla tracciabilità dei prodotti alla gestione dei diritti sulle varietà vegetali, emergono nuove complessità. Le applicazioni esistenti in agricoltura raramente affrontano in modo esplicito il tema della protezione giuridica delle varietà, della gestione delle licenze di coltivazione e del controllo dell'uso autorizzato delle sementi. Di conseguenza, la blockchain viene spesso impiegata come strumento di tracciabilità logistica, più che come supporto alla gestione dei diritti del costitutore.

2.3 Il nostro contributo nel contesto dello stato dell'arte

Alla luce della letteratura analizzata, emerge uno spazio di ricerca e sviluppo nell'integrazione tra blockchain, proprietà intellettuale e gestione delle varietà vegetali. I lavori esistenti mostrano il potenziale della blockchain sia nella registrazione dei diritti di proprietà intellettuale sia nella tracciabilità in ambito agricolo, ma tendono a trattare questi due aspetti in modo separato.

Il contributo di questo lavoro si colloca in questo contesto, proponendo un approccio che mette in relazione la tutela giuridica delle varietà vegetali con la tracciabilità operativa dei lotti di sementi. L'idea di fondo è considerare la blockchain non solo come registro di diritti, ma come strumento di supporto alla gestione delle autorizzazioni e alla verifica della loro applicazione lungo la filiera.

In particolare, il progetto si ispira a modelli accademici esistenti, come il Non-Fungible Plant Variety (NFPV), che evidenziano come la rappresen-

tazione digitale delle varietà possa aumentare trasparenza e controllo. Allo stesso tempo, questo lavoro si inserisce nello stato dell'arte con l'obiettivo di estendere tali concetti verso una gestione più aderente ai processi reali, mantenendo una chiara distinzione tra l'analisi teorica e le scelte progettuali che verranno sviluppate nei capitoli successivi.

Capitolo 3

Metodologia di sviluppo e pianificazione

3.1 Metodologia adottata: Agile–Scrum semplificato

Per lo sviluppo del progetto è stata adottata una metodologia Agile, ispirata al framework Scrum, opportunamente semplificata e adattata al contesto di un progetto universitario di tipo Proof of Concept (PoC). L’approccio Agile è stato scelto per consentire una gestione iterativa del lavoro, favorendo l’evoluzione progressiva del progetto e l’integrazione continua tra analisi teorica, progettazione e sviluppo.

Il progetto è stato organizzato in sprint pianificati, ciascuno caratterizzato da una durata definita e da obiettivi specifici. Gli sprint hanno permesso di suddividere il lavoro in incrementi gestibili, facilitando il controllo dell’avanzamento e la verifica dei risultati intermedi.

In particolare, sono stati pianificati i seguenti sprint:

Sprint 1 – Analisi preliminare e studio dello stato dell’arte

Durata: 1–2 settimane

Obiettivi principali:

- analisi del contesto normativo relativo alla protezione delle varietà vegetali;
- studio dei principali contributi accademici su blockchain e proprietà intellettuale;
- definizione dell’ambito del progetto e degli obiettivi generali.

Sprint 2 – Modellazione concettuale e definizione dei requisiti

Durata: 1–2 settimane

Obiettivi principali:

- individuazione degli attori coinvolti e dei loro ruoli;
- definizione dei requisiti funzionali e non funzionali del sistema;
- prima modellazione dei processi tramite diagrammi UML.

Sprint 3 – Progettazione del sistema e architettura

Durata: 1–2 settimane

Obiettivi principali:

- definizione dell'architettura logica del sistema;
- progettazione dei componenti principali e delle loro interazioni;
- affinamento dei diagrammi UML a supporto della progettazione.

Sprint 4 – Sviluppo del prototipo

Durata: 2 settimane

Obiettivi principali:

- implementazione di un prototipo funzionante del sistema;
- realizzazione delle funzionalità principali previste;
- integrazione tra le diverse componenti del progetto.

Sprint 5 – Verifica, validazione e documentazione finale

Durata: 1 settimana

Obiettivi principali:

- verifica del corretto funzionamento del prototipo;
- valutazione dei risultati ottenuti rispetto agli obiettivi iniziali;
- redazione della documentazione finale e preparazione dell'elaborato.

L'adozione di questo approccio Agile-Scrum semplificato ha consentito di gestire il progetto in modo strutturato ma flessibile, permettendo di affrontare progressivamente le diverse fasi del lavoro e di adattare le attività in base alle esigenze emerse durante lo sviluppo.

3.2 Pianificazione del progetto e diagramma di Gantt

La pianificazione temporale del progetto è rappresentata mediante un diagramma di Gantt, che sintetizza le principali attività e la loro distribuzione nel tempo in relazione agli sprint pianificati. Il diagramma di Gantt del progetto è consultabile all'interno della documentazione allegata al progetto.

3.3 Analisi dei rischi

Nel corso dello sviluppo del progetto sono stati individuati alcuni rischi potenziali, sia di natura tecnica sia di natura organizzativa, che potrebbero influenzare il corretto svolgimento delle attività o il raggiungimento degli obiettivi prefissati. L'analisi dei rischi è stata condotta con l'obiettivo di identificare le principali criticità e di valutarne l'impatto sul progetto.

Un primo insieme di rischi riguarda gli aspetti tecnici. In particolare, l'utilizzo di tecnologie emergenti come la blockchain e gli smart contract comporta una curva di apprendimento iniziale e la possibilità di incontrare limitazioni tecniche non previste nelle fasi preliminari. Tali rischi sono stati mitigati attraverso una fase iniziale di studio approfondito e mediante lo sviluppo incrementale del prototipo.

Un secondo gruppo di rischi è legato agli aspetti temporali e organizzativi. Trattandosi di un progetto con scadenze definite, eventuali ritardi in una fase potrebbero ripercuotersi sulle attività successive. Per ridurre questo rischio, il progetto è stato pianificato in sprint consecutivi, consentendo di monitorare l'avanzamento e di intervenire tempestivamente in caso di scostamenti rispetto alla pianificazione.

Infine, sono stati considerati rischi di natura normativa e concettuale, legati alla corretta interpretazione del contesto giuridico delle varietà vegetali protette. Un'errata comprensione degli aspetti normativi potrebbe compromettere la coerenza del modello proposto. Questo rischio è stato affrontato facendo riferimento alla letteratura accademica e alla normativa internazionale, integrando tali elementi fin dalle fasi iniziali del progetto.

3.4 Stima dei costi

La stima dei costi del progetto è stata effettuata considerando la natura sperimentale del lavoro e il fatto che esso si configuri come un Proof of Concept

sviluppato in ambito universitario. Di conseguenza, la valutazione economica non ha l'obiettivo di fornire una stima industriale dettagliata, ma di dare un'indicazione realistica delle risorse necessarie per la realizzazione del progetto.

I costi principali sono riconducibili al tempo di sviluppo, inteso come impegno complessivo del team nelle attività di analisi, progettazione, implementazione e documentazione. Trattandosi di un progetto accademico, tali costi non vengono quantificati in termini di compenso economico, ma rappresentano comunque la risorsa più rilevante in termini di impegno.

Dal punto di vista infrastrutturale, il progetto fa uso di strumenti e tecnologie open-source o comunque disponibili gratuitamente per scopi didattici, come ambienti di sviluppo, librerie software e piattaforme di test. L'eventuale utilizzo di servizi blockchain o ambienti di esecuzione è limitato a contesti di prova, con costi trascurabili o nulli.

Un elemento di costo potenziale, tipico dei sistemi basati su blockchain, è rappresentato dalle *transaction fees* (gas fees) associate alle operazioni che modificano lo stato on-chain (ad esempio registrazione di una varietà, emissione di una licenza o creazione di un lotto). Nel contesto del PoC tali costi risultano contenuti, poiché le operazioni vengono eseguite in ambiente di test o su reti a basso costo; tuttavia, in uno scenario reale questo aspetto andrebbe considerato in fase di adozione, scegliendo reti e configurazioni in grado di garantire sostenibilità economica per gli attori coinvolti.

Inoltre, la soluzione è stata progettata appositamente per ridurre al minimo il consumo di gas, limitando le scritture on-chain alle sole informazioni essenziali. Un esempio concreto è rappresentato dalle funzioni di tipo *view*, utilizzate per la consultazione e la verifica dei dati: trattandosi di operazioni di sola lettura, non richiedono gas quando eseguite dall'utente, poiché non modificano lo stato della blockchain ma si limitano a visualizzare le informazioni già registrate.

Un'ulteriore scelta progettuale orientata alla riduzione dei costi riguarda la gestione dei documenti: essi non vengono caricati direttamente on-chain, poiché l'inserimento di contenuti di grandi dimensioni incrementerebbe significativamente il costo delle transazioni. Al loro posto, viene registrato in blockchain un hash che garantisce integrità e collegamento al contenuto, mentre il documento viene conservato in uno storage decentralizzato (ad esempio tramite Pinata), rendendolo consultabile senza appesantire la chain e mantenendo contenuti i costi complessivi.

Per la stima del gas è stato utilizzato un tool di Hardhat per il gas reporting, configurato con Solidity 0.8.28, optimizer attivo (200 runs) e block gas limit pari a 30 000 000. Il report ottenuto ha prodotto i risultati riportati in Tabella 3.1.

Tabella 3.1: Risultati del gas report (Hardhat) per le principali funzioni dello smart contract `VarietyLicenseRegistry`.

Funzione	Gas medio	Min	Max	# test
<code>registerVariety</code>	326248	–	–	22
<code>issueLicense</code>	257212	228714	262950	18
<code>createBatch</code>	245200	232192	246584	11
<code>inspectBatch</code>	71885	39059	86830	6
<code>revokeLicense</code>	100652	–	–	2
<code>revokeVariety</code>	51603	51576	51636	4
<code>revokeLicenseByAuthority</code>	96298	–	–	1
<code>updateLicenseExpiration</code>	37502	–	–	1
<code>makeLicensePermanent</code>	37401	–	–	1
<code>addInspector</code>	47810	–	–	12
<code>removeInspector</code>	25976	–	–	1
<code>beginAuthorityTransfer</code>	46297	–	–	1
<code>acceptAuthority</code>	28742	–	–	1

Deployment contratto `VarietyLicenseRegistry`: 4022661 gas.

Infine, non sono previsti costi significativi legati a licenze software proprietarie o a infrastrutture hardware dedicate, poiché il prototipo è stato sviluppato e testato utilizzando risorse già disponibili. Alla luce di queste considerazioni, il costo complessivo del progetto può essere considerato contenuto, coerente con gli obiettivi formativi e con il contesto universitario in cui il lavoro è stato svolto.

Capitolo 4

Requisiti e casi d'uso

4.1 Requisiti funzionali (FURPS – F)

I requisiti funzionali descrivono le funzionalità che il sistema deve offrire ai diversi attori coinvolti, in accordo con i casi d'uso modellati nei diagrammi UML. Ogni requisito è definito in relazione al ruolo specifico dell'attore e alle operazioni che esso è autorizzato a compiere all'interno del sistema.

Requisiti funzionali per l'Authority

L'Authority rappresenta l'ente responsabile della supervisione, del controllo e della validazione delle operazioni critiche del sistema. Il sistema deve consentire all'Authority di:

- validare le richieste di registrazione di nuove varietà vegetali, confermando l'inserimento nel registro una volta completate le verifiche previste;
- gestire gli attori di controllo (ad esempio gli inspector), rendendo possibile l'abilitazione o la rimozione di soggetti incaricati delle verifiche;
- intervenire in modo straordinario su entità critiche (varietà/licenze/lotti) quando necessario per garantire coerenza e rispetto delle regole del sistema;

L'Authority svolge quindi un ruolo di garanzia istituzionale, assicurando che il sistema operi in modo controllato e coerente con le regole di validazione previste.

Requisiti funzionali per il Breeder

Il Breeder (costitutore) è l'attore titolare dei diritti sulle varietà vegetali. Il sistema deve consentire al Breeder di:

- registrare una nuova varietà vegetale protetta, previa validazione dell'Authority;
- consultare l'elenco delle varietà di cui è titolare;
- emettere licenze di coltivazione associate a una varietà registrata;
- revocare o aggiornare licenze precedentemente emesse.

Queste funzionalità permettono al Breeder di gestire l'intero ciclo di vita giuridico della varietà.

Requisiti funzionali per il Licenziatario

Il Licenziatario è il soggetto autorizzato alla coltivazione di una varietà protetta tramite licenza. Il sistema deve consentire al Licenziatario di:

- consultare le licenze di coltivazione a lui associate;
- registrare la creazione di uno o più lotti di produzione, collegandoli a una licenza valida;
- consultare lo stato dei lotti prodotti.

Il Licenziatario interagisce con il sistema principalmente nella fase operativa della produzione.

Requisiti funzionali per l'Inspector

L'Inspector è l'attore incaricato delle attività di controllo e verifica. Il sistema deve consentire all'Inspector di:

- verificare la legittimità di varietà, licenze e lotti di produzione;
- registrare l'esito delle ispezioni effettuate;
- consultare lo storico delle ispezioni.

Il ruolo dell'Inspector consente di collegare le informazioni registrate nel sistema con le verifiche effettuate nel mondo reale.

Requisiti funzionali per il Pubblico

Il Pubblico rappresenta l'insieme degli utenti che accedono al sistema in modalità consultiva. Il sistema deve consentire al Pubblico di:

- verificare l'autenticità e la legittimità di un lotto di sementi;
- consultare informazioni essenziali relative alla varietà e alla licenza associata;
- accedere alle informazioni in modo semplice e senza necessità di autenticazione.

Queste funzionalità garantiscono trasparenza e favoriscono la fiducia nei confronti del sistema.

4.2 Requisiti non funzionali (URPS)

I requisiti non funzionali descrivono le caratteristiche qualitative del sistema e i vincoli che ne determinano il comportamento complessivo. Essi non definiscono le funzionalità offerte, ma specificano le proprietà che il sistema deve possedere affinché tali funzionalità risultino efficaci, affidabili e utilizzabili nel contesto applicativo di riferimento.

Nel presente progetto, i requisiti non funzionali sono organizzati secondo il modello URPS, che comprende usabilità, affidabilità, prestazioni e trasportabilità.

Usability (Usabilità)

L'usabilità rappresenta la capacità del sistema di essere facilmente utilizzabile e comprensibile da parte degli utenti. Considerata la presenza di attori eterogenei, con competenze tecniche differenti, il sistema deve risultare accessibile anche a utenti non esperti di tecnologie informatiche o blockchain.

In particolare, il sistema deve:

- fornire interfacce chiare e intuitive per la consultazione delle informazioni;
- consentire la verifica delle informazioni principali tramite operazioni semplici e immediate;
- ridurre al minimo il numero di passaggi necessari per completare le operazioni più comuni;

- presentare i dati in modo comprensibile, evitando ambiguità o interpretazioni errate.

Reliability (Affidabilità)

L'affidabilità indica la capacità del sistema di mantenere informazioni corrette, coerenti e persistenti nel tempo. Questo requisito è particolarmente rilevante poiché il sistema gestisce dati sensibili legati alla tutela delle varietà vegetali protette.

Il sistema deve:

- garantire l'integrità delle informazioni relative a varietà, licenze e lotti;
- mantenere uno storico completo e non alterabile delle operazioni effettuate;
- prevenire modifiche non autorizzate ai dati critici;
- assicurare che lo stato delle entità gestite rifletta sempre la situazione aggiornata.

Performance (Prestazioni)

Le prestazioni riguardano la capacità del sistema di rispondere alle richieste in tempi adeguati e con un utilizzo efficiente delle risorse disponibili. Pur essendo concepito come Proof of Concept, il sistema deve risultare adeguato a un utilizzo operativo realistico.

In particolare, il sistema deve:

- consentire verifiche rapide di varietà, licenze e lotti;
- garantire tempi di risposta accettabili per le operazioni di consultazione;
- mantenere un comportamento stabile in presenza di un numero moderato di richieste concorrenti.

Supportability (Supporto e manutenibilità)

La supportabilità riguarda la facilità con cui il sistema può essere mantenuto, aggiornato ed esteso nel tempo. Questo aspetto è fondamentale per consentire l'evoluzione del sistema in risposta a nuove esigenze applicative o a cambiamenti del contesto normativo.

Il sistema deve:

- essere strutturato in modo modulare;
- facilitare l'introduzione di nuovi attori o nuovi casi d'uso;
- consentire l'adattamento a differenti contesti normativi;
- permettere l'aggiornamento delle funzionalità senza compromettere la stabilità complessiva del sistema.

4.3 Casi d'uso dettagliati

I casi d'uso descrivono le principali interazioni tra gli attori e il sistema e consentono di comprendere il comportamento atteso nelle diverse fasi operative. Essi sono stati modellati mediante diagrammi UML dei casi d'uso, che evidenziano gli attori coinvolti e le funzionalità accessibili a ciascuno di essi, fornendo una rappresentazione chiara delle dinamiche applicative.

Nei diagrammi UML condivisi, gli attori principali sono l'Authority, il Breeder, il Licenziatario, l'Inspector e il Pubblico, ciascuno con un insieme ben definito di interazioni con il sistema.

Caso d'uso: Registrazione di una varietà vegetale

Il caso d'uso della registrazione di una varietà vegetale, rappresentato nel relativo diagramma UML dei casi d'uso, coinvolge principalmente il Breeder come attore iniziatore e l'Authority con un ruolo di validazione.

Il processo ha inizio quando il Breeder richiede la registrazione di una nuova varietà, inserendo le informazioni necessarie alla sua identificazione. Il sistema acquisisce tali dati e ne verifica la completezza formale. Successivamente, come mostrato nel diagramma dei casi d'uso, l'Authority interviene per validare la richiesta. Una volta completata la validazione, la varietà viene registrata e associata in modo univoco al Breeder.

Eccezioni e alternative. Un'eccezione a questo flusso si verifica nel caso in cui la validazione da parte dell'Authority non abbia esito positivo. In tale scenario, la registrazione viene rifiutata e la varietà non risulta disponibile nel sistema. Il diagramma UML evidenzia questa alternativa come percorso distinto rispetto al flusso principale.

Caso d'uso: Emissione della licenza di coltivazione

Il caso d'uso dell'emissione della licenza di coltivazione è rappresentato nei diagrammi UML come un'azione eseguita dal Breeder su una varietà precedentemente registrata.

Il Breeder seleziona una varietà e richiede l'emissione di una licenza, specificandone le condizioni. Il sistema registra la licenza e la rende disponibile per il soggetto autorizzato. Questo caso d'uso costituisce il collegamento tra la tutela giuridica della varietà e la successiva attività produttiva.

Eccezioni e alternative. Un'alternativa prevista nel diagramma dei casi d'uso riguarda la revoca o la modifica di una licenza. In questo caso, il Breeder può intervenire su una licenza già emessa, modificandone lo stato. Il sistema deve gestire correttamente tali variazioni, aggiornando lo stato associato alle licenze interessate.

Caso d'uso: Creazione di un lotto di produzione

La creazione di un lotto di produzione è un caso d'uso che coinvolge il Licenziatario, come mostrato nel diagramma UML dei casi d'uso.

Il processo inizia quando il Licenziatario, in possesso di una licenza valida, registra un nuovo lotto di produzione. Il sistema verifica che la licenza associata sia effettivamente valida e attiva, e successivamente collega il lotto alla licenza stessa. Il lotto diventa così tracciabile e consultabile all'interno del sistema.

Eccezioni e alternative. Un'eccezione si verifica quando il Licenziatario tenta di registrare un lotto utilizzando una licenza non valida, revocata o scaduta. In questo caso, come evidenziato nel diagramma dei casi d'uso, il sistema rifiuta la creazione del lotto, impedendo l'associazione a una licenza non autorizzata.

Caso d'uso: Verifica del lotto

Il caso d'uso della verifica del lotto è rappresentato nei diagrammi UML come un'interazione accessibile a più attori, in particolare all'Inspector e al Pubblico, seppur con livelli di accesso differenti.

L'Inspector utilizza il sistema per verificare la legittimità di un lotto di produzione, consultando le informazioni relative alla varietà e alla licenza associata e registrando l'esito dell'ispezione. Parallelamente, il Pubblico può

accedere alle funzionalità di consultazione per verificare l'autenticità di un lotto, ottenendo informazioni essenziali sul suo stato.

Eccezioni e alternative. Un'alternativa a questo flusso si verifica quando il lotto risulta associato a una licenza revocata o non valida. In tali casi, il sistema segnala l'anomalia e rende evidente lo stato irregolare del lotto, come previsto nei diagrammi dei casi d'uso.

Capitolo 5

Architettura e tech stack

5.1 Architecture overview (3 layer)

L'architettura proposta è organizzata in tre livelli con responsabilità ben separate: frontend layer (DApp), blockchain layer e off-chain storage layer.

Nel frontend layer, l'applicazione decentralizzata fornisce interfacce dedicate ai diversi attori (Authority, Breeder, Licensee, Inspector e Pubblico), consentendo l'esecuzione dei casi d'uso tramite un'interazione guidata e comprensibile anche a utenti non specialisti. Il frontend comunica con la blockchain attraverso una Web3 API, invocando le funzioni dello smart contract e mostrando feedback sulle transazioni.

Il blockchain layer costituisce la componente di verità condivisa del sistema. Qui risiede lo smart contract principale, responsabile della gestione dello stato delle entità core (varietà, licenze e lotti), dei controlli di accesso per ruolo e dell'emissione di eventi utili a tracciare le operazioni nel tempo.

Infine, l'off-chain storage layer è utilizzato per memorizzare contenuti e documentazione che non è opportuno mantenere direttamente on-chain (ad esempio documenti ufficiali in PDF, immagini e metadati estesi), mantenendo nel ledger solo riferimenti verificabili (hash e URI) utili a garantire integrità e collegamento tra dato digitale e risorsa esterna. È importante notare che, nella modellazione attuale, non tutte le entità on-chain includono riferimenti off-chain: i campi per hash/URI sono presenti per la sola entità *Variety*, mentre *License* e *Batch* non prevedono riferimenti dedicati. Qualora si volesse associare documentazione off-chain anche a licenze e lotti, sarebbe necessario estendere le strutture dati e la logica dello smart contract.

5.2 Tech stack dettagliato

Il sistema è implementato utilizzando uno stack tecnologico coerente con l'obiettivo di realizzare un Proof of Concept completo, mantenendo però strumenti e librerie standard del panorama DApp. Lo stack on-chain/off-chain/frontend descritto risulta coerente con la configurazione effettiva presente nel repository.

Blockchain

- **Linguaggio:** Solidity 0.8.28
- **Framework di sviluppo:** Hardhat ^2.28 (con toolbox)
- **Rete di sviluppo:** Hardhat local node (localhost)
- **Tooling incluso:** toolbox con supporto a ethers v6, TypeChain, gas-reporter e coverage

Hardhat include gli strumenti per misurare costi e qualità del codice, ma è utile specificare che:

- il **gas reporter non è abilitato di default**, quindi il report viene generato solo se configurato esplicitamente (ad esempio con la sezione `gasReporter` e/o la variabile `REPORT_GAS`);
- la **coverage** non viene prodotta automaticamente, ma richiede l'esecuzione esplicita di `npx hardhat coverage`.

Backend / Off-chain services

Il backend off-chain è implementato come servizio Node + Express e gestisce upload, validazione e integrazione con storage decentralizzato tramite Pinata.

- **Runtime (container backend):** Node.js 18
- **Framework:** Express
- **Upload e gestione file:** multer
- **Validazione input:** express-validator
- **CORS:** cors
- **Configurazione:** variabili d'ambiente tramite `.env`

- **IPFS integration:** Pinata SDK (@pinata/sdk) tramite moduli dedicati (es. servizi IPFS)

Frontend

- **Bundler:** Vite
- **Framework UI:** React 18.x
- **Web3 library:** ethers.js 6.9
- **HTTP client:** axios
- **Wallet:** MetaMask (browser extension)
- **UI framework:** Tailwind CSS 4.x
- **QR:** qrcode.react
- **Scansione QR:** html5-qrcode
- **Icone:** lucide-react
- **Routing:** React Router 6.x
- **E2E testing:** Playwright

Nota di versioning: la containerizzazione del frontend utilizza Node 20 per build e preview (Dockerfile frontend), mentre lo stack applicativo rimane coerente con React/Vite dichiarati.

DevOps / Containerizzazione

- **Docker Compose:** orchestrazione di 3 servizi (Hardhat node, backend Express, frontend Vite) su rete bridge dedicata (plantnet), con variabili configurabili per API/RPC/IPFS gateway
- **Dockerfile dedicati** per frontend e backend

5.3 Smart contract architecture

Lo smart contract principale (`VarietyLicenseRegistry`) modella e gestisce l'intero workflow applicativo: registrazione varietà → emissione licenze → creazione lotti → ispezione/verifica. Dal punto di vista strutturale, la modellazione si basa su enum per descrivere gli stati, struct per rappresentare le entità del dominio e mapping per indicizzare e recuperare i dati in modo efficiente.

In particolare, gli stati fondamentali del sistema sono rappresentati da enumerazioni (ad esempio stati della varietà, della licenza, del lotto e dell'ispezione), oltre a un meccanismo di trust level per qualificare l'esito della filiera informativa.

Le entità principali (*Variety*, *License*, *Batch*, *Inspection*) includono campi identificativi, riferimenti agli attori coinvolti e, dove previsto, collegamenti a documenti off-chain tramite hash e URI, così da mantenere on-chain solo ciò che è critico e verificabile. Nell'implementazione corrente, il collegamento off-chain è esplicitato a livello di *Variety* (tramite `documentHash` e `documentURI`), mentre per *License* e *Batch* la tracciabilità è principalmente garantita attraverso riferimenti e identificativi on-chain. Un'estensione per includere metadati/documenti anche su *License* e *Batch* richiederebbe un aggiornamento delle strutture dati e delle funzioni del contratto.

Il contratto espone un insieme di funzioni pubbliche coerenti con i casi d'uso (ad esempio registrare varietà, emettere/revocare licenze, creare lotti, registrare esiti di ispezione e verificare lo stato), e genera eventi per rendere osservabili le transizioni di stato e facilitare l'aggiornamento della DApp.

5.4 Pattern architetturali e sicurezza

L'architettura applica pattern consolidati per garantire chiarezza progettuale, coerenza del modello e sicurezza operativa. Tra i principali pattern adottati rientrano:

- **Access Control Pattern:** uso di modifier per vincolare le operazioni critiche ai ruoli autorizzati.
- **Factory Pattern:** creazione controllata di varietà, licenze e lotti tramite funzioni dedicate.
- **Event-Driven Pattern:** eventi on-chain per tracking, audit trail e aggiornamento della UI.

- **Oracle Pattern:** integrazione dell'attore Inspector come ponte tra verifiche reali e registrazione digitale.
- **Mapping Pattern:** storage efficiente e accesso rapido alle entità indicizzate.

Sul piano della sicurezza, il sistema adotta misure coerenti con le best practice degli smart contract: controllo accessi, validazione degli input, emissione di eventi per auditabilità e prevenzione di vulnerabilità note (ad esempio attenzione a chiamate esterne non sicure). Inoltre, l'uso di Solidity 0.8+ consente la gestione nativa di overflow/underflow.

Capitolo 6

Prototipo

All'interno della documentazione del progetto sono disponibili i diagrammi UML (casi d'uso, sequenza e altri diagrammi di supporto) che descrivono i flussi operativi e le interazioni tra DApp, storage off-chain e smart contract.

6.1 Implementazione smart contract

Il prototipo è basato su uno smart contract principale, pensato per rappresentare l'intero ciclo di vita di una varietà vegetale protetta: dalla registrazione iniziale, passando per l'emissione delle licenze, fino alla creazione, ispezione e verifica pubblica dei lotti.

A livello implementativo, il contratto (identificabile come `VarietyLicenseRegistry`) mantiene uno stato on-chain strutturato tramite contatori (ID progressivi) e mapping, così da avere uno storage efficiente e facilmente interrogabile. Le funzioni core sono progettate seguendo una logica “a permessi”, cioè ogni operazione critica è eseguibile solo dall'attore autorizzato (Authority, Breeder, Licenziatario, Inspector).

Le principali funzioni implementate e il loro ruolo nel sistema sono:

- **`registerVariety(...)`**: consente all'Authority di registrare una nuova varietà inserendo dati identificativi e collegando un documento ufficiale tramite:
 - hash del PDF (per garantire integrità),
 - URI del documento salvato off-chain (per evitare di caricare file pesanti in blockchain).

La logica prevede controlli su permessi e unicità (es. numero di registrazione già esistente). Il completamento genera un evento on-chain che la DApp può usare per ricostruire lo storico e/o aggiornare le viste.

- **issueLicense(...)**: permette al Breeder (proprietario della varietà) di emettere una licenza verso un licenziatario, con eventuale data di scadenza. Il contratto valida che:
 - la varietà sia attiva,
 - il chiamante sia davvero il proprietario,
 - la scadenza (se presente) non sia nel passato,
 - non esista già una licenza attiva duplicata per lo stesso licenziatario.

Anche qui l'esito produce eventi utili per auditabilità e ricostruzione della timeline.

- **createBatch(...)**: consente al Licenziatario di creare un lotto associato a una licenza valida, includendo quantità e metadati. Prima della creazione vengono verificati:
 - titolarità della licenza,
 - stato “attivo” della licenza,
 - non scadenza / non revoca,
 - quantità > 0.

In caso di successo viene emesso l'evento di creazione lotto e la DApp può generare un QR code collegato al **batchId**.

- **inspectBatch(batchId, approve)**: (nel contratto reale è una funzione unica) riservata all'Inspector autorizzato. L'Inspector può registrare l'esito di una verifica fisica passando un booleano **approve** (**true/false**). Rispetto a una modellazione con due funzioni separate (**approve/reject**), questa implementazione:
 - non raccoglie una “motivazione di rifiuto” on-chain (non esiste un campo **reason**);
 - consente re-ispezioni, cioè un lotto può essere ispezionato più volte nel tempo.

Lo smart contract continua comunque a gestire vincoli fondamentali (es. autorizzazione dell'Inspector e coerenza dello stato), e rende l'ispezione tracciabile tramite eventi e dati registrati nello storage.

- **verifyBatch(batchId)**: è una funzione di sola lettura (**view**) pensata per la verifica pubblica: dato l’ID del lotto, restituisce un responso verificabile senza costi di transazione. Nel contratto attuale è importante notare che:
 - se il **batchId** non esiste, la funzione non restituisce un “esito negativo”, ma fa **revert** perché è protetta da un controllo (modifier) di esistenza del lotto; la DApp gestisce questo caso intercettando l’errore e mostrando all’utente “lotto non trovato”;
 - la logica include controlli aggiuntivi, ad esempio il caso di produzione antecedente alla registrazione della varietà;
 - l’esito non è solo “valido/non valido”, ma costruisce un *trust level* che tiene conto anche dell’eventuale ispezione e della coerenza temporale tra eventi (licenza, produzione, revoche/scadenze).

6.2 Frontend – DApp

Il prototipo include una DApp web che consente agli attori di interagire con lo smart contract attraverso un’interfaccia a ruoli. L’obiettivo è rendere le operazioni principali fruibili senza richiedere competenze specifiche di blockchain, delegando al frontend la gestione della UX (form, validazioni, messaggi, navigazione) e lasciando alla blockchain la funzione di “fonte di verità”.

Nel flusso operativo, la DApp:

- recupera lo stato on-chain con chiamate di lettura (**view**);
- prepara le transazioni per le operazioni che modificano lo stato (registrazione, emissione licenze, creazione lotti, ispezioni);
- usa un wallet (es. MetaMask) per firma e invio.

Rispetto a una DApp che mantiene listener attivi sugli eventi, l’implementazione nel repository non registra listener permanenti per aggiornare automaticamente la UI: l’aggiornamento avviene tramite ricaricamenti mirati con chiamate dirette al contratto e, quando serve ricostruire lo storico, tramite query dei log (ad esempio usando **queryFilter**) invocate “al bisogno” dalle pagine.

Le funzionalità principali, lato interfaccia, sono organizzate in tre macro-aree:

Authority Dashboard

- registrazione varietà (con caricamento documento e collegamento hash/URI);
- consultazione elenco varietà registrate;
- gestione ispettori (abilitazione/rimozione);
- panoramica e statistiche di sistema (in forma prototipale).

Inspector Dashboard

- lista lotti;
- dettaglio lotto con dati disponibili e storico;
- azione di ispezione tramite la funzione `inspectBatch(approve=true/false)`.

In allineamento con lo smart contract, la UI:

- non richiede né conserva una motivazione di rifiuto;
- consente re-ispezioni (il lotto può essere ispezionato anche più volte).

Verifica pubblica

- scansione QR (mobile) o input manuale `batchId`;
- risposta “a semaforo” basata sugli esiti di `verifyBatch`;
- gestione dell’errore “lotto inesistente” intercettando il `revert` della `view`;
- accesso ai dettagli della varietà e al documento collegato quando disponibile.

6.3 Sequence diagrams & eventi

Per collegare in modo chiaro comportamento dell’app e implementazione, i diagrammi di sequenza evidenziano come frontend, storage off-chain e smart contract cooperano, includendo anche i flussi alternativi (errori, annullamenti, `revert`). In questa sezione è utile sottolineare che, nel prototipo, gli eventi on-chain sono utilizzati soprattutto per auditabilità e per ricostruire la timeline tramite query dei log, mentre l’aggiornamento della UI avviene principalmente tramite chiamate dirette e refresh mirati.

Registrazione varietà

Il flusso mostra un processo completo con gestione errori:

- l'Authority compila i dati e carica un PDF;
- la DApp valida input e calcola l'hash del documento;
- il file viene salvato in storage off-chain e si ottiene un URI;
- l'utente firma la transazione e invoca `registerVariety(...)`.
- lo smart contract applica controlli (permessi, unicità, validità dati);
- in caso di successo vengono emessi eventi utili per storico e tracciamento.

Sono esplicitati anche casi alternativi: input incompleto, fallimento storage, annullamento MetaMask, chiamante non autorizzato, varietà duplicata.

Emissione licenza e creazione lotto

Qui si vede la concatenazione tra due azioni fondamentali:

- il Breeder emette una licenza (`issueLicense`) con controlli su proprietà, stato varietà e scadenza;
- il licenziatario, visualizzate le licenze attive, crea un lotto (`createBatch`) con ulteriori vincoli (quantità valida, licenza attiva, non revocata, non scaduta).

Il flusso include eccezioni realistiche (indirizzo non valido, varietà non attiva, licenza già presente, licenza scaduta/revocata, titolarità errata), e conclude con la creazione del lotto e generazione del QR.

Ispezione lotto

Il flusso descrive il ruolo dell'Inspector nella chiusura del "gap" tra fisico e digitale:

- l'ispettore seleziona un lotto;
- registra l'esito usando la funzione unica `inspectBatch(batchId, approve)`;
- il contratto verifica i permessi dell'Inspector e registra lo stato;

- l'operazione è tracciata tramite eventi e/o aggiornamento dello stato on-chain.

In coerenza con l'implementazione:

- non viene gestita una motivazione di rifiuto on-chain;
- è possibile che un lotto venga ispezionato più volte (re-ispezione), rendendo lo storico più aderente a controlli ripetuti.

Verifica lotto pubblica

La verifica pubblica è modellata come chiamata **view** (nessuna transazione, nessun costo), con una logica di controllo approfondita. Nel contratto reale però:

- se il lotto non esiste, la chiamata fa **revert** (non ritorna un valore); la DApp deve quindi gestire l'eccezione e mostrare l'esito "lotto non trovato";
- la funzione include controlli aggiuntivi, come la coerenza temporale tra registrazione varietà, emissione licenza e produzione lotto;
- l'esito finale costruisce un trust level che tiene conto anche dell'ispezione, oltre a revoke/scadenze e ordine temporale degli eventi.

Il flusso può inoltre mostrare come la DApp ricostruisce una timeline tramite eventi (es. emissione licenza, creazione lotto) usando query mirate, presentando poi l'esito con un semaforo di trust.

Capitolo 7

Validazione e verifica

7.1 Test unitari smart contract

La validazione del prototipo è stata avviata partendo dai test unitari sullo smart contract, con l'obiettivo di verificare che ogni funzione rispetti i vincoli di ruolo, le precondizioni e le transizioni di stato previste. I test sono stati realizzati in ambiente Hardhat, utilizzando il framework di testing integrato (Mocha/Chai), con un approccio “per casi d'uso”, cioè costruendo suite dedicate a registrazione varietà, gestione licenze, lotti, ispezioni e verifica pubblica.

Un esempio significativo riguarda la registrazione di una varietà, dove viene verificato che l'operazione sia consentita solo all'Authority e che l'esecuzione produca l'evento atteso. Nel test viene poi controllato lo stato on-chain per assicurarsi che i dati salvati siano coerenti con l'input. In parallelo viene testato il caso negativo, in cui un attore non autorizzato tenta la stessa operazione e la transazione deve andare in revert con messaggio di errore coerente.

Oltre ai flussi principali, la suite copre anche i casi limite (edge cases) più rilevanti per il dominio: duplicazioni, stati non validi (es. varietà revocata o licenza non valida), azioni ripetute (inclusa la re-ispezione dei lotti), input incompleti e tentativi di accesso da ruoli errati. L'esecuzione complessiva dei test fornisce un riscontro positivo in termini di esiti, con una suite ampia che copre la maggior parte delle funzionalità del contratto.

7.2 Test end-to-end e UI/UX

Accanto ai test unitari, è stata effettuata una verifica end-to-end (E2E) dell'intero flusso applicativo, simulando l'interazione reale tra i diversi attori e

la DApp. Lo scopo è controllare non solo la correttezza logica, ma anche la solidità dell'integrazione tra frontend, wallet e smart contract, e la chiarezza dei feedback verso l'utente.

Per la parte frontend, i test E2E sono stati implementati con Playwright, in modo da automatizzare scenari realistici di navigazione, compilazione dei form, invio di operazioni e gestione degli errori. In particolare, sono stati verificati i punti critici tipici delle DApp: stabilità della connessione al wallet, validazione dei form, gestione dei revert e feedback durante le transazioni.

Lo scenario E2E principale segue il ciclo "dalla registrazione alla verifica", coinvolgendo i ruoli chiave: registrazione varietà, emissione licenza, creazione lotto, ispezione e verifica pubblica. Sono stati testati anche percorsi alternativi, ad esempio annullamento della firma, inserimento di un identificativo inesistente nella verifica pubblica, tentativo di creazione lotto con licenza non valida e ispezione ripetuta sullo stesso lotto.

7.3 Analisi coverage e strumenti di test

La verifica è stata completata con un'analisi orientata alla copertura funzionale dei test e alla qualità complessiva della suite.

Per la parte blockchain, l'esecuzione dei test in Hardhat produce un output strutturato che documenta in modo chiaro le suite testate e i risultati ottenuti. In particolare, viene riportato un riepilogo delle aree funzionali coperte (gestione ruoli e funzioni amministrative, varietà, licenze, lotti, ispezioni e verifica), insieme ai risultati dell'esecuzione e alla copertura complessiva ottenuta. Il dettaglio dei risultati di coverage e dei test eseguiti è consultabile all'interno della documentazione allegata al progetto.

Capitolo 8

Discussione

8.1 Scelte progettuali principali

Una prima scelta chiave riguarda il modo in cui il sistema rappresenta l'affidabilità delle informazioni lungo la filiera. Invece di adottare una logica binaria (valido/non valido), abbiamo modellato un approccio a livelli di trust, così da riflettere meglio i casi reali: un lotto può essere tecnicamente registrato, ma non ancora ispezionato, oppure collegato a una licenza scaduta o revocata. Questo rende la verifica più utile per l'utente finale, perché non restituisce solo un "sì/no", ma un'informazione graduata e contestualizzata.

Un'altra scelta rilevante è stata separare in modo netto ciò che deve stare on-chain da ciò che può stare off-chain. Gli elementi che richiedono integrità, tracciabilità e non ripudiabilità (registrazione varietà, emissione/revoca licenze, creazione lotti, approvazioni ispettive) vengono gestiti dallo smart contract, mentre documenti e descrizioni estese restano fuori catena e vengono collegati tramite hash/URI. In questo modo la blockchain resta la "fonte di verità" per lo stato logico del sistema, senza appesantire inutilmente lo storage.

A livello di modello applicativo, una scelta caratterizzante è l'inserimento esplicito dell'Inspector come attore "ponte" tra mondo fisico e digitale. Nei diagrammi UML (casi d'uso e sequence diagram) questa dinamica è centrale: il sistema non si limita a registrare dati, ma include un passaggio di validazione qualificata che aumenta la credibilità del lotto quando deve essere verificato dal pubblico.

Infine, sul piano implementativo, le scelte di stack e workflow (sviluppo, test, deploy) sono state orientate alla realizzazione di un PoC robusto e verificabile, privilegiando strumenti che facilitano test automatici, copertura dei casi critici e tracciabilità delle evoluzioni del codice.

8.2 Sfide incontrate e soluzioni adottate

Sfida 1 – Collegamento affidabile tra eventi reali e dati on-chain (oracle problem). Il problema principale, comune a molte applicazioni blockchain “fisiche”, è garantire che ciò che viene scritto in blockchain corrisponda a un fatto reale (esistenza del lotto, provenienza, rispetto della licenza, ecc.). La mitigazione adottata nel PoC è duplice:

da un lato, hash e riferimenti ai documenti permettono di verificare che un documento non sia stato alterato;

dall'altro, l'introduzione dell'Inspector consente di registrare on-chain un atto di validazione legato a una verifica nel mondo reale, come mostrato nei flussi UML dedicati all'approvazione dei lotti.

Sfida 2 – Costi e latenza delle transazioni. Le operazioni on-chain non sono “gratuite” e introducono tempi legati alla conferma delle transazioni. Per un PoC è importante dimostrare la fattibilità senza esplodere in costi o complessità. La soluzione è stata progettare funzioni e strutture dati essenziali, mantenendo in catena solo ciò che serve davvero e usando letture “view” dove possibile (verifica pubblica), in modo da rendere la consultazione leggera e veloce.

Sfida 3 – Trasparenza vs. privacy. La blockchain è trasparente per natura, ma la filiera agricola può includere informazioni sensibili (dati commerciali, documenti, dettagli operativi). Per questo la progettazione evita di mettere on-chain contenuti estesi o dati che non siano necessari alla verifica: l'utente pubblico deve poter controllare l'autenticità e lo stato di un lotto, senza ottenere informazioni che non sono utili allo scopo o che potrebbero essere inappropriate da esporre.

Sfida 4 – Scalabilità e gestione dei dati. La filiera genera molti eventi: licenze, lotti, ispezioni, verifiche. Salvare tutto on-chain in modo dettagliato non è sostenibile. Il compromesso adottato è usare la blockchain come registro dello stato e degli eventi critici, e demandare a componenti off-chain (storage e indicizzazione lato applicazione) la parte descrittiva e documentale. Questo approccio è coerente con l'architettura mostrata nei diagrammi, dove l'applicazione funge da strato di orchestrazione tra smart contract e storage.

8.3 Limitazioni e sviluppi futuri

Pur essendo efficace come Proof of Concept, la soluzione presenta limiti importanti.

Il primo è l'adozione istituzionale e la valenza legale operativa: il sistema può aumentare trasparenza e tracciabilità, ma non “sostituisce automaticamente” i registri ufficiali o le procedure formali. In un contesto reale, la blockchain funzionerebbe al meglio come registro complementare, integrato con enti e processi già esistenti.

Un secondo limite è legato alla centralità dell'Authority: se l'Authority è un singolo punto decisionale, si introduce una dipendenza organizzativa che riduce la decentralizzazione effettiva. Uno sviluppo naturale è introdurre meccanismi di governance più robusti (es. multi-firma, ruoli distribuiti, audit trail organizzativo più forte), così da aumentare resilienza e fiducia.

Terzo punto: il sistema dipende anche dal fattore umano (Inspector). Se l'ispettore sbaglia o agisce in malafede, la blockchain registra comunque quell'azione. Per questo, in un'evoluzione futura, sarebbe utile integrare: criteri di autorizzazione più stringenti, logiche di reputazione/monitoraggio, firme digitali e controlli incrociati (anche tra più ispettori) sui lotti più critici.

Dal lato utente, un limite pratico è l'esperienza d'uso legata ai wallet (es. MetaMask). Per un'applicazione destinata anche a utenti non tecnici, uno sviluppo importante è migliorare l'autenticazione e la UX (account abstraction, login più familiari, gestione guidata delle transazioni), mantenendo però la separazione dei privilegi tra ruoli come modellato nei casi d'uso.

Infine, in ottica di prodotto reale, restano aperti sviluppi tecnici rilevanti: audit e hardening dei contratti, integrazione con standard e sistemi esterni, e l'estensione del modello per gestire scenari più complessi (più tipologie di controlli, certificazioni, eccezioni normative, storicizzazione avanzata delle ispezioni).

Capitolo 9

Conclusioni e sviluppi futuri

9.1 Obiettivi raggiunti

L'obiettivo del progetto era dimostrare la fattibilità di un sistema basato su blockchain per la gestione di varietà vegetali protette, in grado di coprire l'intero workflow: registrazione della varietà, emissione e revoca delle licenze, creazione dei lotti, ispezione e verifica pubblica. Il Proof of Concept sviluppato raggiunge i seguenti risultati principali.

In primo luogo, è stato realizzato uno smart contract completo e funzionante, che modella le entità fondamentali del dominio (varietà, licenze, lotti e ispezioni) e implementa le operazioni chiave coerenti con i casi d'uso. L'impiego sistematico degli eventi consente di tracciare le transizioni di stato nel tempo e di supportare l'interazione della DApp con la blockchain in modo robusto e verificabile.

In secondo luogo, il prototipo include interfacce operative dedicate ai diversi ruoli (Authority, Breeder, Licenziatario, Inspector e Pubblico), garantendo una chiara separazione delle responsabilità. Questo aspetto è rilevante perché il sistema non è concepito come una piattaforma generica di tracciabilità, ma come una soluzione centrata sulla gestione dei diritti del costituente e sulla verifica dei lotti lungo la filiera.

Un ulteriore contributo è rappresentato dall'adozione di una logica di verifica non puramente binaria. Il sistema gestisce scenari realistici introducendo una valutazione graduata dell'affidabilità delle informazioni (trust level) e includendo l'Inspector come attore di validazione capace di collegare eventi reali e registrazioni digitali. Questo permette di trattare correttamente casi come lotti non ancora ispezionati o situazioni in cui licenze scadono o vengono revocate dopo la produzione.

Infine, la validazione tramite test unitari e test end-to-end ha fornito

evidenze di coerenza tra requisiti, casi d'uso e implementazione, confermando la correttezza dei principali flussi operativi e la solidità dell'integrazione tra DApp, wallet e smart contract.

9.2 Sviluppi futuri

Nonostante i risultati ottenuti, la soluzione può essere estesa e migliorata in diverse direzioni, con interventi progressivi su aspetti tecnici, organizzativi e normativi.

Sviluppi a breve termine. Un primo obiettivo consiste nel consolidare il prototipo rendendolo più facilmente dimostrabile in contesti reali. Tra gli interventi utili rientrano il deploy su una rete di test pubblica, il miglioramento dell'esperienza utente (in particolare riducendo la complessità legata alle firme e alla gestione del wallet) e l'introduzione di funzionalità di supporto come notifiche per eventi critici e strumenti di monitoraggio più avanzati.

Sviluppi a medio termine. In un secondo momento, sarebbe opportuno aumentare la robustezza complessiva del sistema. La governance dell'Authority potrebbe essere rafforzata con meccanismi di controllo più resilienti (come multi-signature). Inoltre, un'estensione rilevante riguarda l'affidabilità degli Inspector: in scenari reali, può essere utile introdurre logiche di reputazione, audit e tracciamento delle ispezioni per ridurre l'impatto del fattore umano.

Sviluppi a lungo termine. In un orizzonte più ampio, il sistema potrebbe evolvere verso interoperabilità e privacy avanzata. L'interoperabilità con registri esterni e con altre soluzioni blockchain aumenterebbe l'adozione in contesti multi-ente o transnazionali. Parallelamente, tecniche crittografiche come le Zero-Knowledge Proofs potrebbero consentire verifiche robuste mantenendo riservate informazioni che non devono essere pubbliche.

Aspetti legali e compliance. Per un'adozione reale è necessario considerare vincoli normativi importanti, in particolare la gestione di dati potenzialmente personali e la compatibilità con principi di tutela della privacy. In questo senso, un'evoluzione naturale consiste nel mantenere on-chain solo informazioni strettamente necessarie e nel gestire contenuti modificabili o rimovibili esclusivamente off-chain. Inoltre, la blockchain va considerata come strumento complementare rispetto a registri ufficiali e procedure istituzionali: il suo valore principale è aumentare verificabilità e auditabilità, non sostituire automaticamente i processi legali esistenti.

Bibliografia

- [1] UPOV, *International Convention for the Protection of New Varieties of Plants (UPOV Convention)*, Act of 1991, UPOV Publication No. 221(E), Geneva, 1991.
- [2] M. Rapela, L. Lehtinen, *Non-Fungible Plant Variety (NFPV): A Proposal for an Innovative Way of Controlling Seed Trade of Protected Plant Varieties*, International Journal of Innovative Science and Research Technology, vol. 8, no. 1, pp. 1357–1367, 2023.
- [3] J. Hugendubel, *Blockchain Technology and Intellectual Property – A Basic Introduction*, SSRN Electronic Journal, 2021.